

Projet_2025

Malek

Les bibliothèques utiles pour le projet

```
library(mlbench)
library(ggplot2)
library(GGally)
library(MASS)
library(e1071)
```

```
##
## Attachement du package : 'e1071'

## L'objet suivant est masqué depuis 'package:ggplot2':
##
##     element
```

```
library(caret)
```

```
## Le chargement a nécessité le package : lattice
```

```
library(dplyr)
```

```
##
## Attachement du package : 'dplyr'

## L'objet suivant est masqué depuis 'package:MASS':
##
##     select

## Les objets suivants sont masqués depuis 'package:stats':
##
##     filter, lag

## Les objets suivants sont masqués depuis 'package:base':
##
##     intersect, setdiff, setequal, union
```

Etude de simulation :

Nous simulons 2 gaussiennes en dimension 2 avec $\Sigma = I$, $\mu_1^T = (0, 0)$ et $\mu_2^T = (\epsilon, \epsilon)$.
lorsque les deux classes sont de proportion identique $\Rightarrow \pi_1 = \pi_2$

```

simul <- function(eps,n_obs) {
  n1 = n_obs/2

  var = diag(2) # la variance de nos 2 gaussiennes.

  X1 = mvrnorm(n1,c(0,0),var)
  X2 = mvrnorm(n1,c(eps,eps),var)
  X = rbind(X1,X2) #matrices des données
  Y = c(rep(1,n1),rep(2,n1)) #vecteurs des étiquettes

  return (list(X = X, Y = Y))
}

```

1) ERREUR EMPIRIQUE

```

err_empirique <- function (eps,n_obs,n_sim){
  erreur = numeric(n_sim)
  for (i in 1:n_sim){
    De <- simul(eps,n_obs) #Jeu d'entrainement pour le modèle.
    Dt <- simul(eps,n_obs) #Jeu de test pour le modèle.

    cb <- lda(De$X,De$Y) #cb pour classifieur de bayes, dans le cas de 2 lois gaussiennes de même matric
    res_cb <- predict(cb,Dt$X)$class #les classes de chaque'une des observations estimer par le classifi

    erreur[i] <- mean(res_cb != Dt$Y)
  }
  return (mean(erreur))
}

#Test
print(err_empirique(0.1,200,2025))

```

```
## [1] 0.4845481
```

```
print(err_empirique(1,200,2025))
```

```
## [1] 0.2416321
```

```
print(err_empirique(2,200,2025))
```

```
## [1] 0.07943704
```

```
print(err_empirique(10,200,2025))
```

```
## [1] 0
```

On remarque que plus \$epsilon\$ augmente plus (de façon extrêmement plus rapide l'erreur chute), resul

2) ERREUR THEORIQUE

a) un classifieur de bayes est de la forme :

$$\hat{f} = \begin{cases} 1 & P(X|Y=1) * P(Y=1) > P(X|Y=0) * P(Y=0) \\ 2 & \text{sinon} \end{cases}$$

on pose

$$\pi_1 = P(Y=1)$$

$$\pi_2 = P(Y=2)$$

La densité d'une loi gaussienne multivariée est :

$$\log(P(X|Y=1)) = -(1/2) \log(2\pi|\Sigma|) - (1/2)(x - \mu_1)^T \Sigma^{-1} (x - \mu_1) \text{ idem pour } Y=2$$

En soustrayant les deux équations :

$$\log(P(X|Y=1)) - \log(P(X|Y=2)) = -(1/2)(x - \mu_1)^T \Sigma^{-1} (x - \mu_1) + (1/2)(x - \mu_2)^T \Sigma^{-1} (x - \mu_2)$$

$$\Leftrightarrow \log(P(X|Y=1)) - \log(P(X|Y=2)) = \frac{1}{2} [(x - \mu_2)^T \Sigma^{-1} (x - \mu_2) - (x - \mu_1)^T \Sigma^{-1} (x - \mu_1)]$$

$$\text{on a; } (x - \mu_i)^T \Sigma^{-1} (x - \mu_i) = x^T \Sigma^{-1} x - 2x^T \Sigma^{-1} \mu_i + \mu_i^T \Sigma^{-1} \mu_i$$

$$\Rightarrow \log(P(X|Y=1)) - \log(P(X|Y=2)) = (1/2)(x^T \Sigma^{-1} x - 2x^T \Sigma^{-1} \mu_2 + \mu_2^T \Sigma^{-1} \mu_2 - x^T \Sigma^{-1} x + 2x^T \Sigma^{-1} \mu_1 - \mu_1^T \Sigma^{-1} \mu_1)$$

$$\text{et donc; } \log(P(X|Y=1)) - \log(P(X|Y=2)) = (1/2)(2x^T \Sigma^{-1} (\mu_1 - \mu_2) + \mu_2^T \Sigma^{-1} \mu_2 - \mu_1^T \Sigma^{-1} \mu_1)$$

$$\Leftrightarrow \log(P(X|Y=1)) - \log(P(X|Y=2)) = x^T \Sigma^{-1} (\mu_1 - \mu_2) + (1/2)(\mu_2^T \Sigma^{-1} \mu_2 - \mu_1^T \Sigma^{-1} \mu_1)$$

$$(\text{Par symétrie de } \Sigma) \Rightarrow \log(P(X|Y=1)) - \log(P(X|Y=2)) = (\mu_1 - \mu_2)^T \Sigma^{-1} x - (1/2)(\mu_1^T \Sigma^{-1} \mu_1 - \mu_2^T \Sigma^{-1} \mu_2)$$

On peut factoriser le second terme :

$$\mu_1^T \Sigma^{-1} \mu_1 - \mu_2^T \Sigma^{-1} \mu_2 = (\mu_1 - \mu_2)^T \Sigma^{-1} (\mu_1 + \mu_2)$$

et finalement on a

$$\log(P(X|Y=1)) - \log(P(X|Y=2)) = (\mu_1 - \mu_2)^T \Sigma^{-1} (x - (1/2)(\mu_1 + \mu_2))$$

b) Si X appartient à la classe 1

Si $X \sim N(\mu_1, \Sigma)$, alors:

$$A = (\mu_1 - \mu_2)^T \Sigma^{-1} (X - (1/2)(\mu_1 + \mu_2))$$

on sait que A suit une loi normale car elle est une transformation affine d'une gaussienne, il nous reste donc à calculer son espérance et sa variance.

Sachant $\Sigma = I$

$$E[A] = (\mu_1 - \mu_2)^T (\mu_1 - (1/2)(\mu_1 + \mu_2)) = (1/2) \|\mu_1 - \mu_2\|^2 = \epsilon^2$$

$$Var(A) = (\mu_1 - \mu_2)^T \Sigma (\mu_1 - \mu_2) = \|\mu_1 - \mu_2\|^2 = 2\epsilon^2$$

Donc: $A \sim N(\epsilon^2, 2\epsilon^2)$

c) ERREUR THEORIQUE

```

erreur_th <- function(eps) {
  if (eps == 0) {
    return (0.5)
  }
  moy = eps^2
  var = 2*eps^2
  erreur_classe1 <- pnorm(0,mean = moy,sd = sqrt(var))
  erreur_classe2 <- erreur_classe1
  return ((0.5)*erreur_classe1 + 0.5*erreur_classe2)
}

#test
erreur_th(1)

```

```
## [1] 0.2397501
```

Vos données

```

data(PimaIndiansDiabetes)
df = PimaIndiansDiabetes
head(df)

```

```

##   pregnant glucose pressure triceps insulin mass pedigree age diabetes
## 1         6      148       72      35        0 33.6    0.627  50      pos
## 2         1       85       66      29        0 26.6    0.351  31      neg
## 3         8     183       64       0        0 23.3    0.672  32      pos
## 4         1       89       66      23       94 28.1    0.167  21      neg
## 5         0     137       40      35     168 43.1    2.288  33      pos
## 6         5     116       74       0        0 25.6    0.201  30      neg

```

```
summary(df)
```

```

##      pregnant      glucose      pressure      triceps
##  Min.   : 0.000   Min.   : 0.0   Min.   : 0.00   Min.   : 0.00
## 1st Qu.: 1.000   1st Qu.: 99.0   1st Qu.: 62.00   1st Qu.: 0.00
##  Median : 3.000   Median :117.0   Median : 72.00   Median :23.00
##  Mean   : 3.845   Mean   :120.9   Mean   : 69.11   Mean   :20.54
## 3rd Qu.: 6.000   3rd Qu.:140.2   3rd Qu.: 80.00   3rd Qu.:32.00
##  Max.   :17.000   Max.   :199.0   Max.   :122.00   Max.   :99.00
##      insulin      mass      pedigree      age      diabetes
##  Min.   : 0.0   Min.   : 0.00   Min.   :0.0780   Min.   :21.00   neg:500
## 1st Qu.: 0.0   1st Qu.:27.30   1st Qu.:0.2437   1st Qu.:24.00   pos:268
##  Median : 30.5   Median :32.00   Median :0.3725   Median :29.00
##  Mean   : 79.8   Mean   :31.99   Mean   :0.4719   Mean   :33.24
## 3rd Qu.:127.2   3rd Qu.:36.60   3rd Qu.:0.6262   3rd Qu.:41.00
##  Max.   :846.0   Max.   :67.10   Max.   :2.4200   Max.   :81.00

```

```
str(df)
```

```
## 'data.frame':   768 obs. of  9 variables:
```

```
## $ pregnant: num 6 1 8 1 0 5 3 10 2 8 ...
## $ glucose : num 148 85 183 89 137 116 78 115 197 125 ...
## $ pressure: num 72 66 64 66 40 74 50 0 70 96 ...
## $ triceps : num 35 29 0 23 35 0 32 0 45 0 ...
## $ insulin : num 0 0 0 94 168 0 88 0 543 0 ...
## $ mass : num 33.6 26.6 23.3 28.1 43.1 25.6 31 35.3 30.5 0 ...
## $ pedigree: num 0.627 0.351 0.672 0.167 2.288 ...
## $ age : num 50 31 32 21 33 30 26 29 53 54 ...
## $ diabetes: Factor w/ 2 levels "neg","pos": 2 1 2 1 2 1 2 1 2 2 ...
```

```
dim(df) #768x9
```

```
## [1] 768 9
```

Présentation du jeu de données :

On a 768 observations de femmes d'origine indienne Pima

on 1 variable expliquée : le diabète

Il y a 8 variables explicatives :

-nombre de fois enceintes

-le glucose

-La pression artérielle

-triceps (indicateur de masse grasse sous-cutanée)

-l'insuline

-mass (IMC indice de masse corporelle)

-pedigree

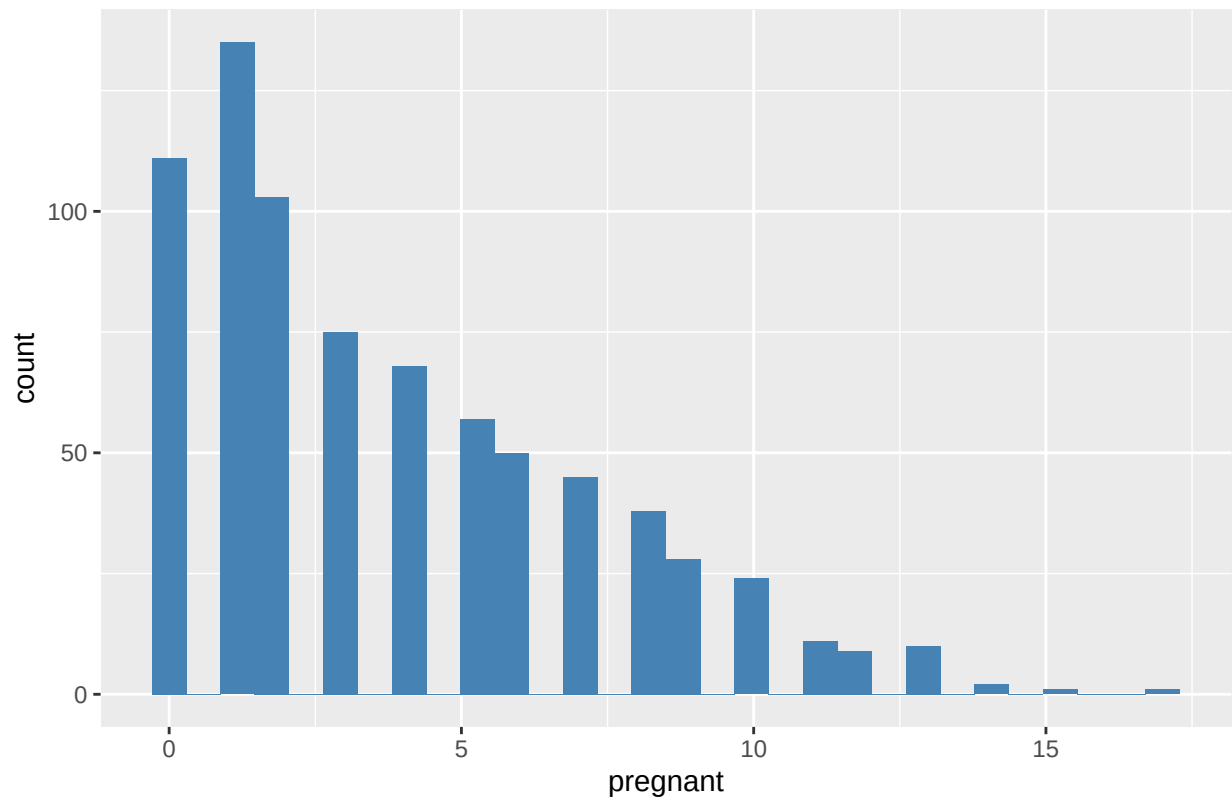
-âge

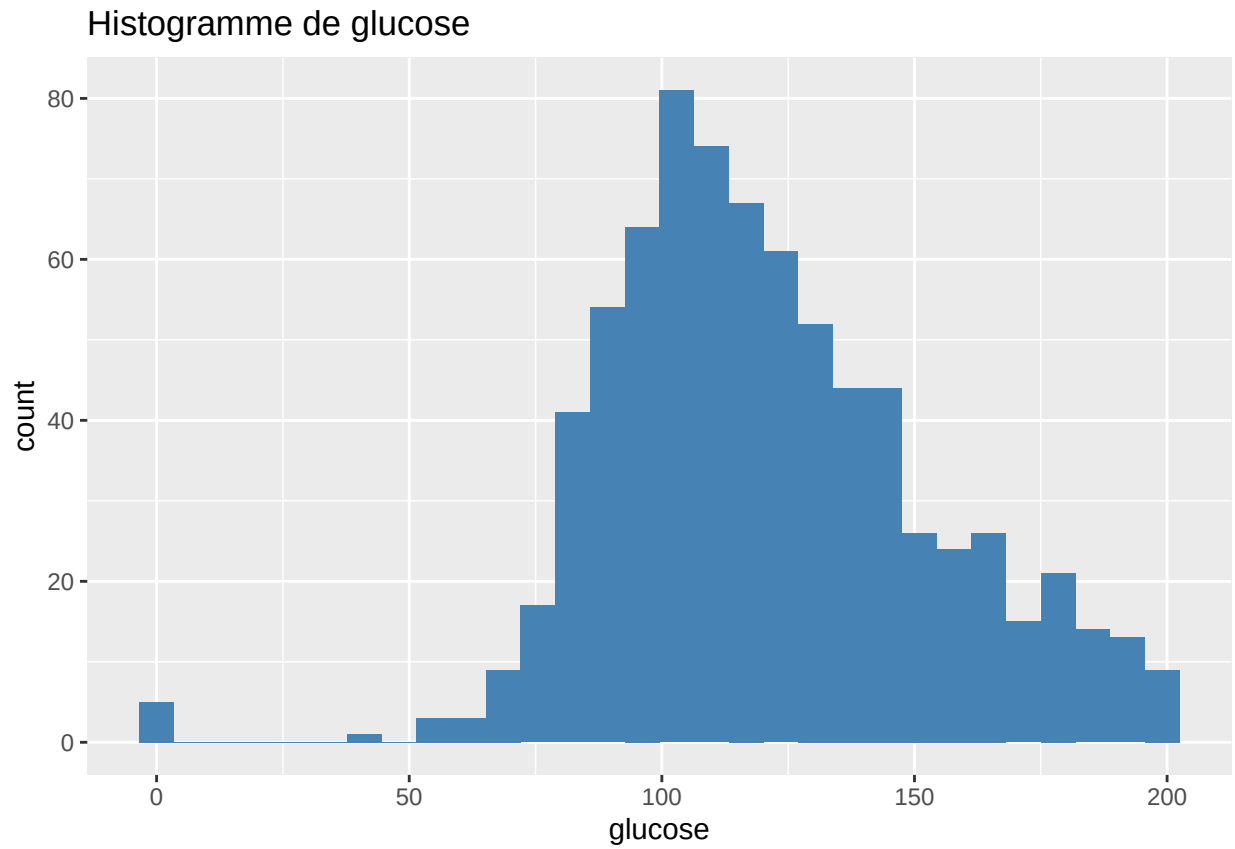
```
#Histogramme méthode univariées}
```

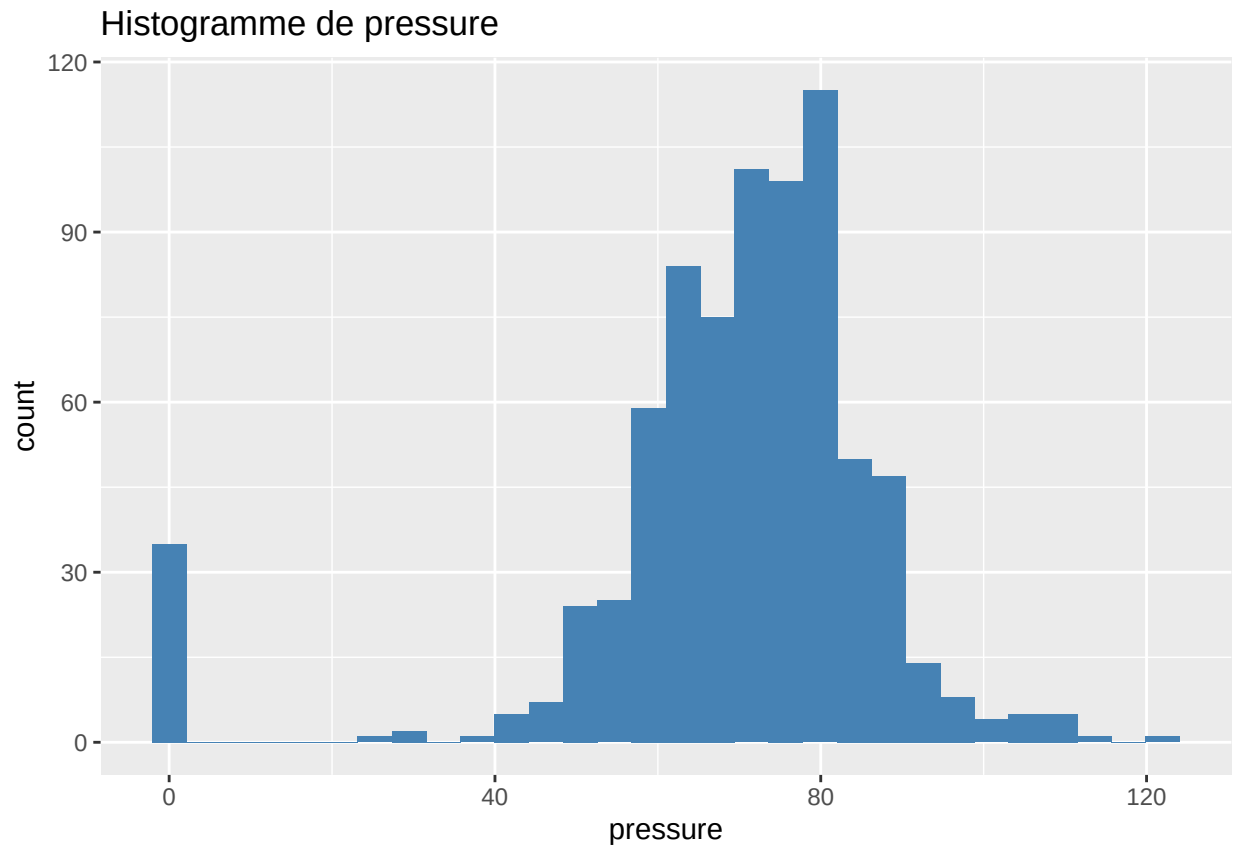
```
for (col in names(df)[1:8]) { print(ggplot(df, aes(x = .data[[col]])) +  
  ggtitle(paste("Histogramme de", col))) }
```

```
geom_histogram(bin
```

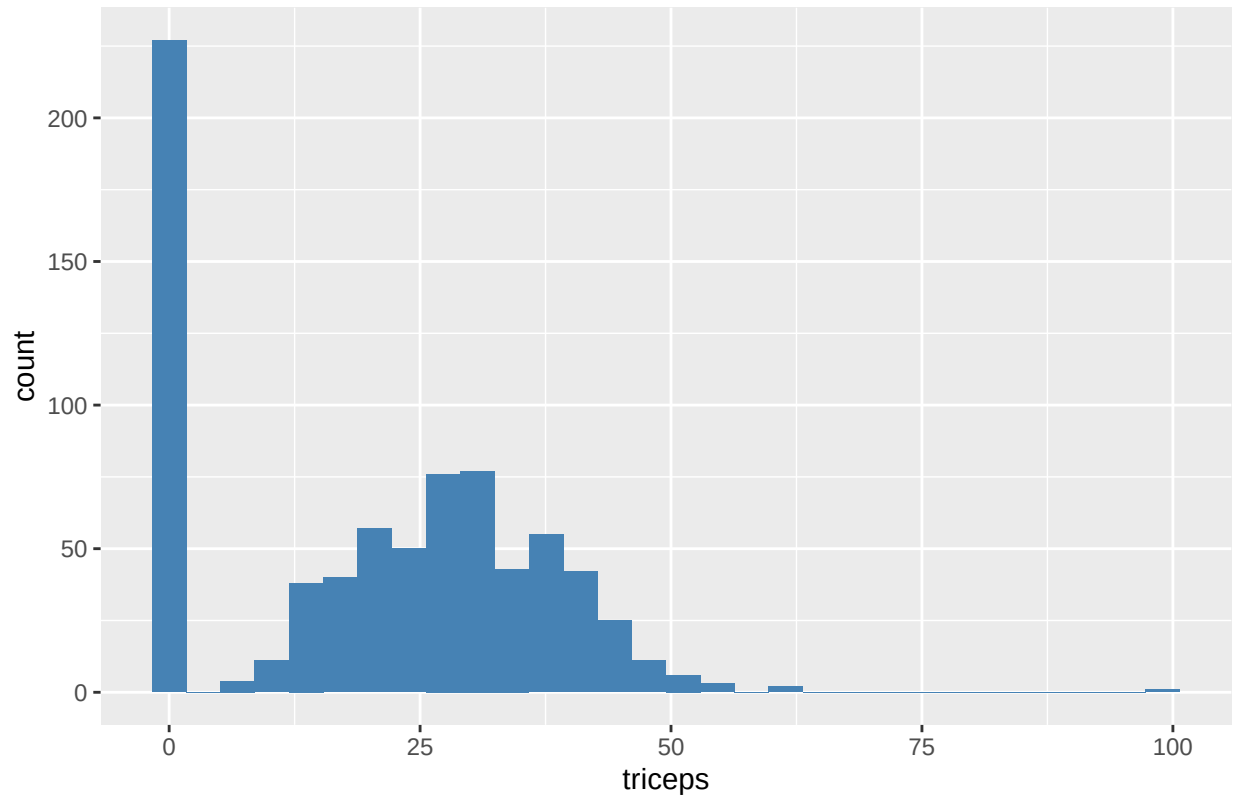
Histogramme de pregnant



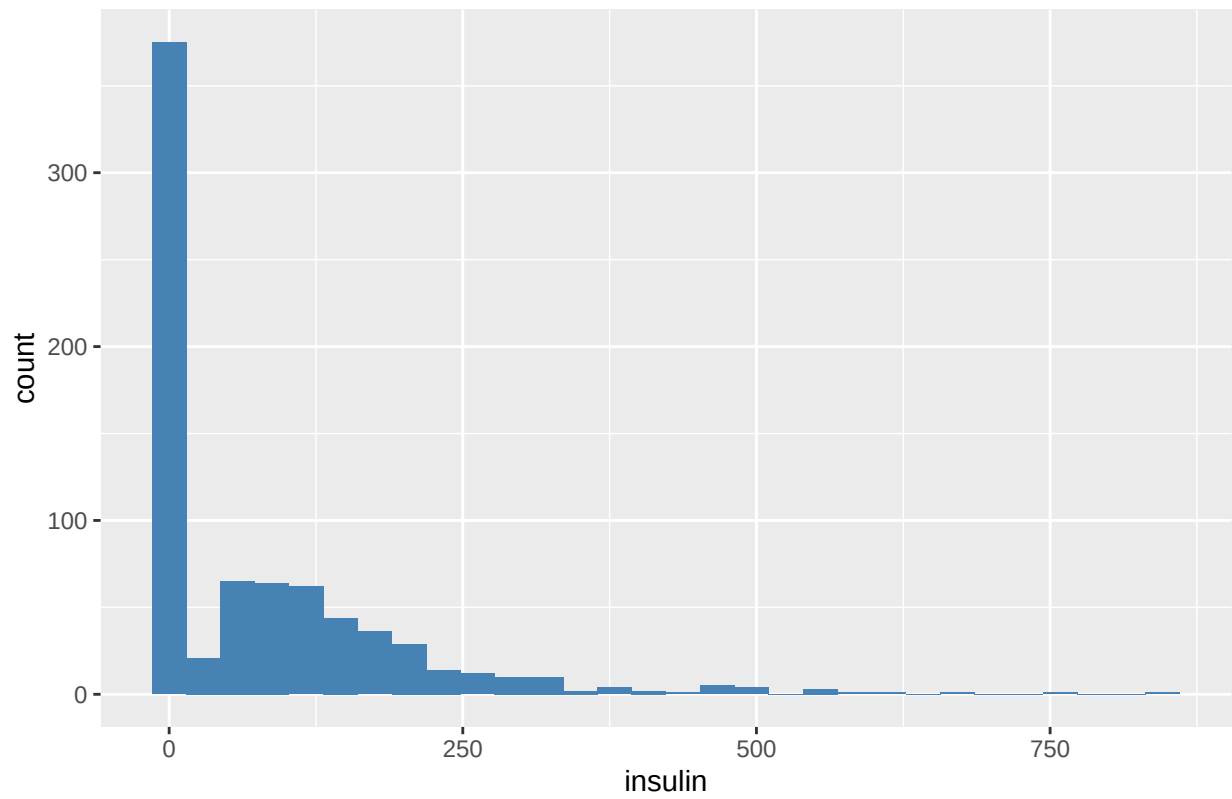


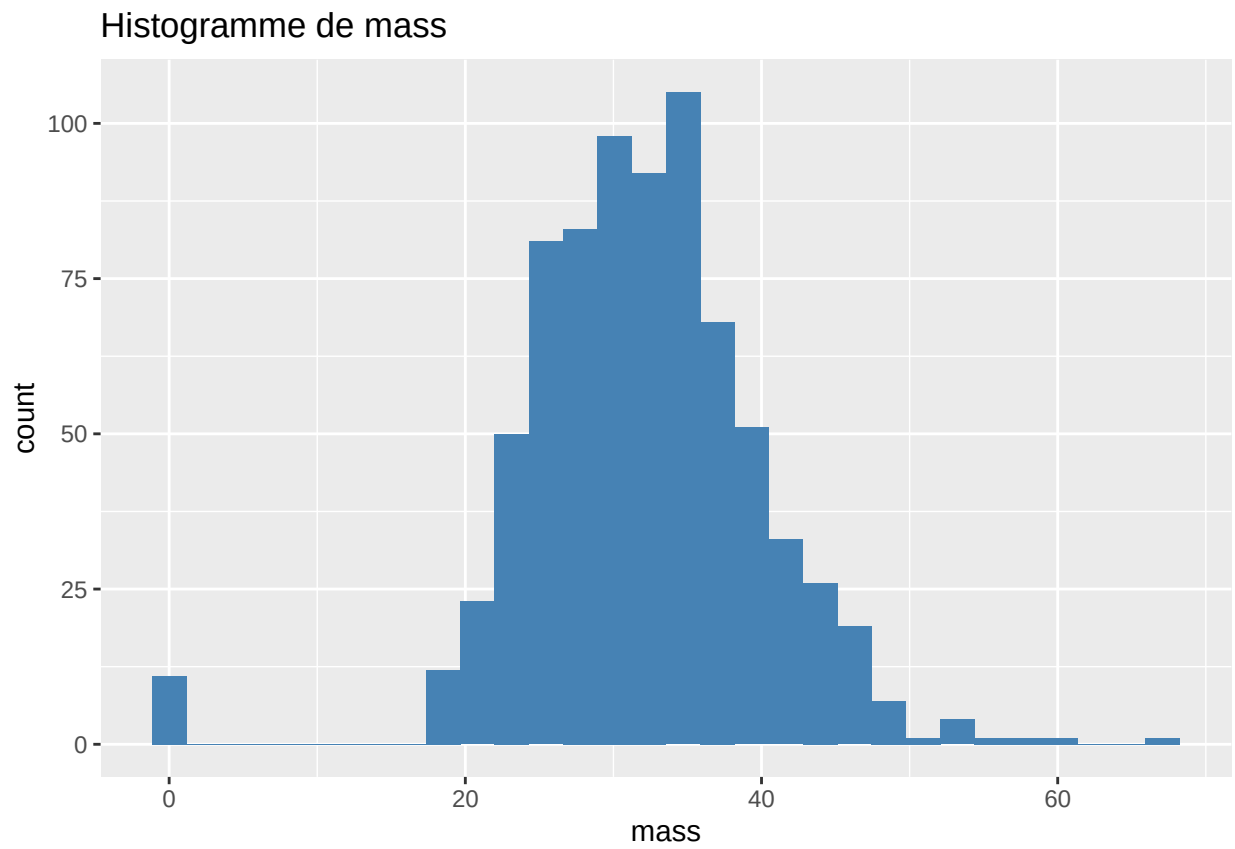


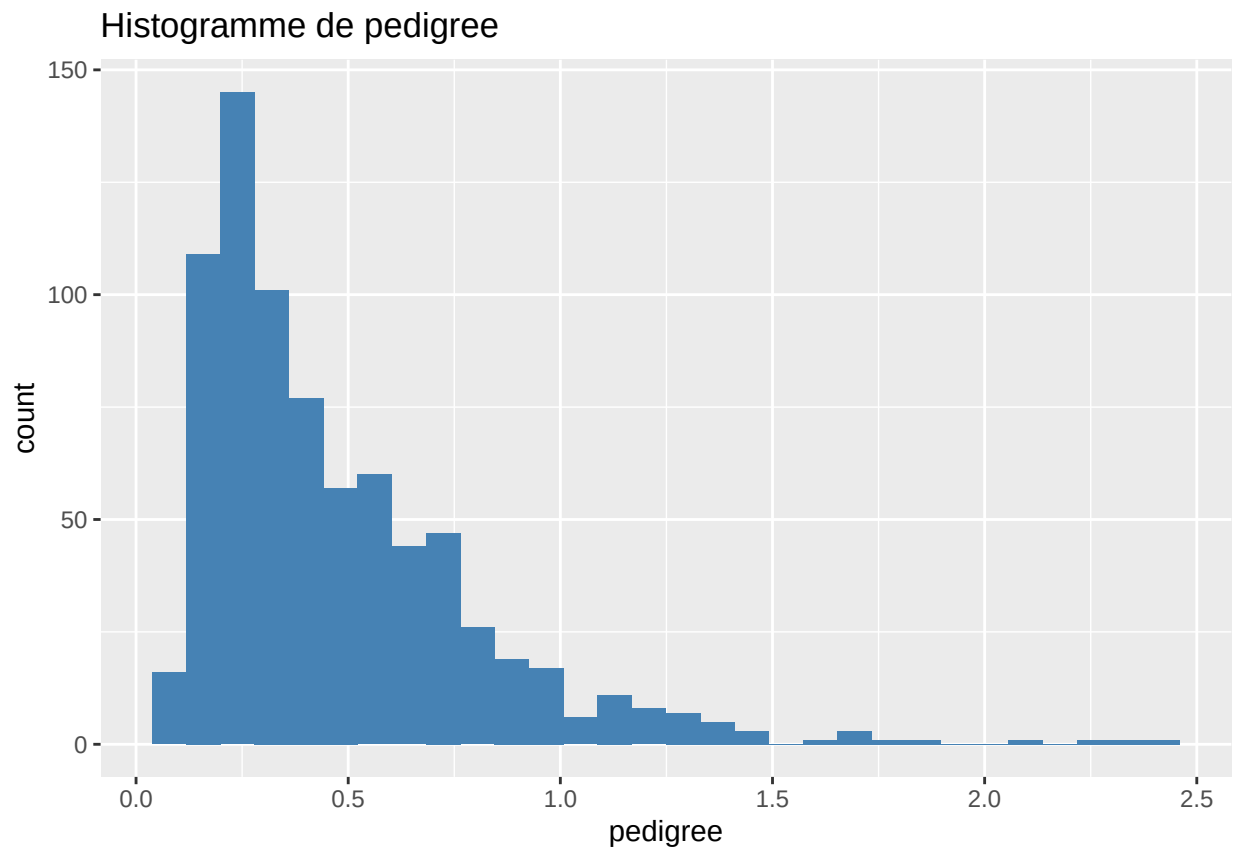
Histogramme de triceps



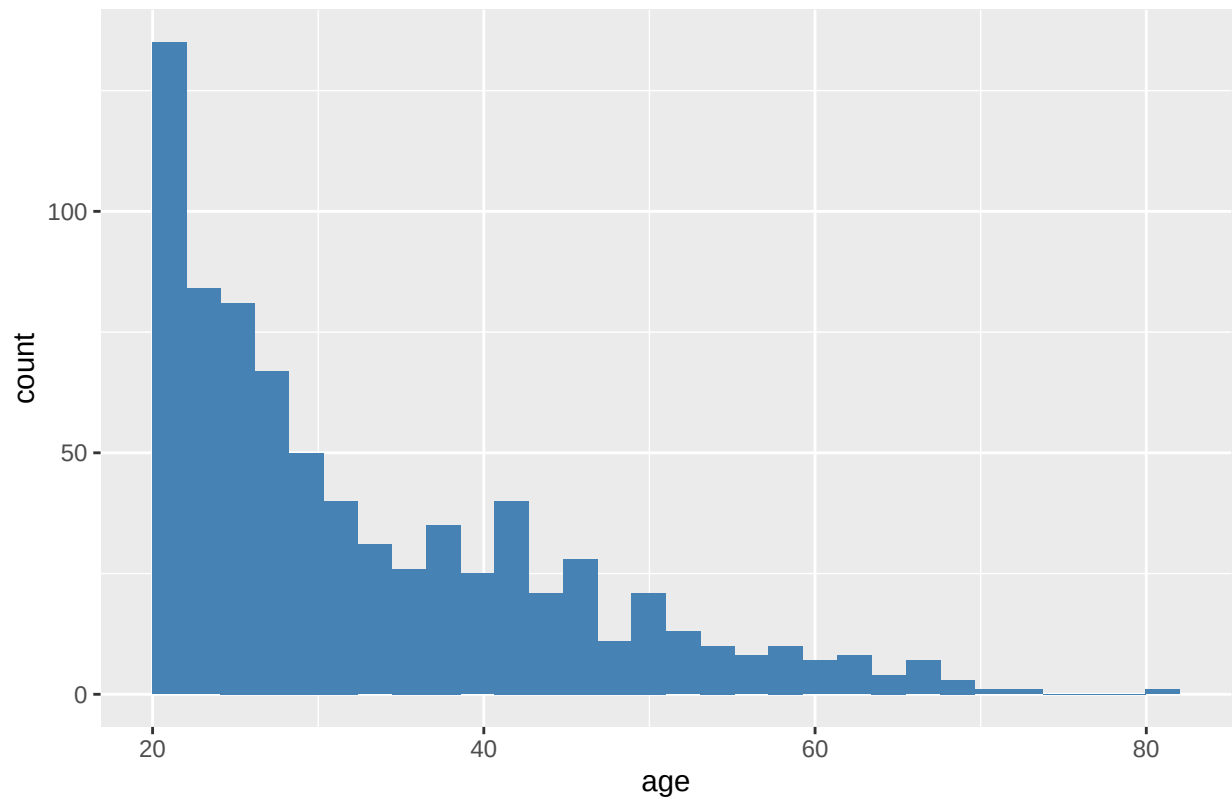
Histogramme de insulin



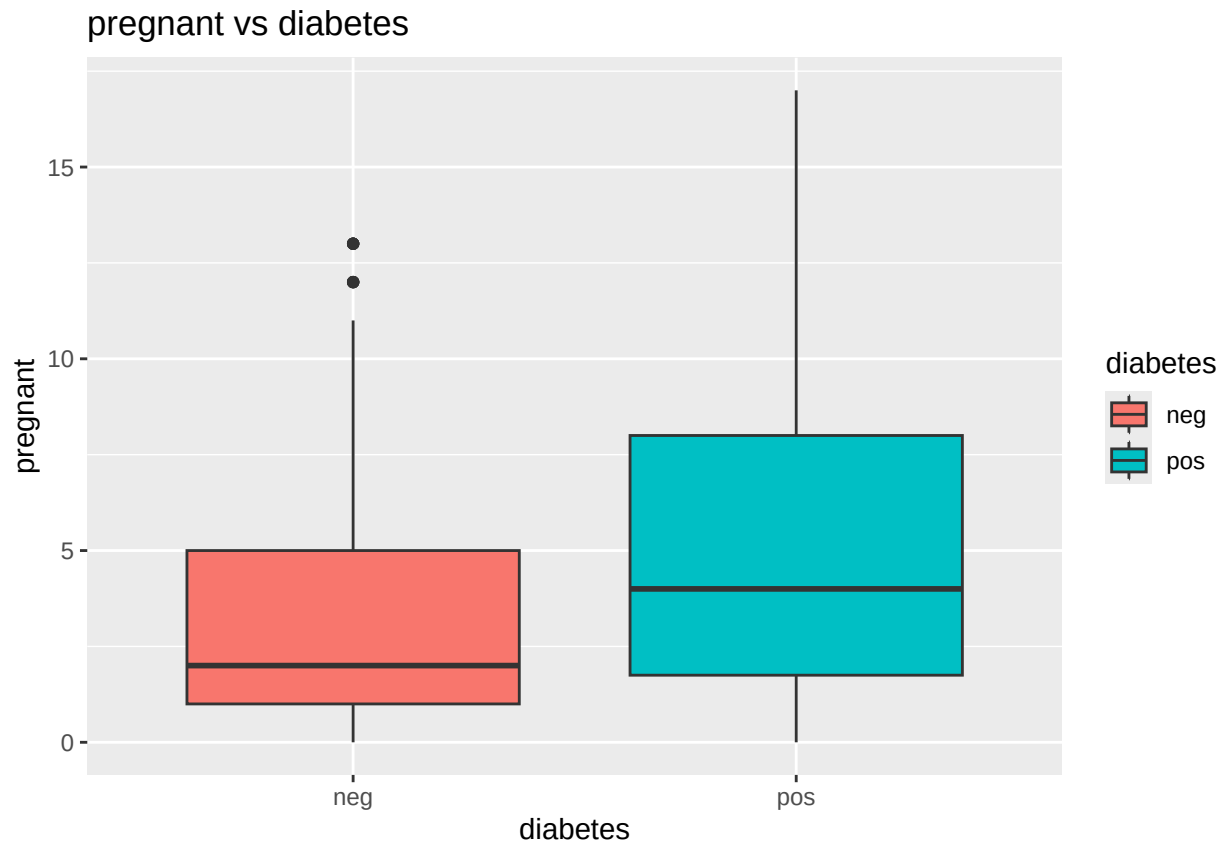


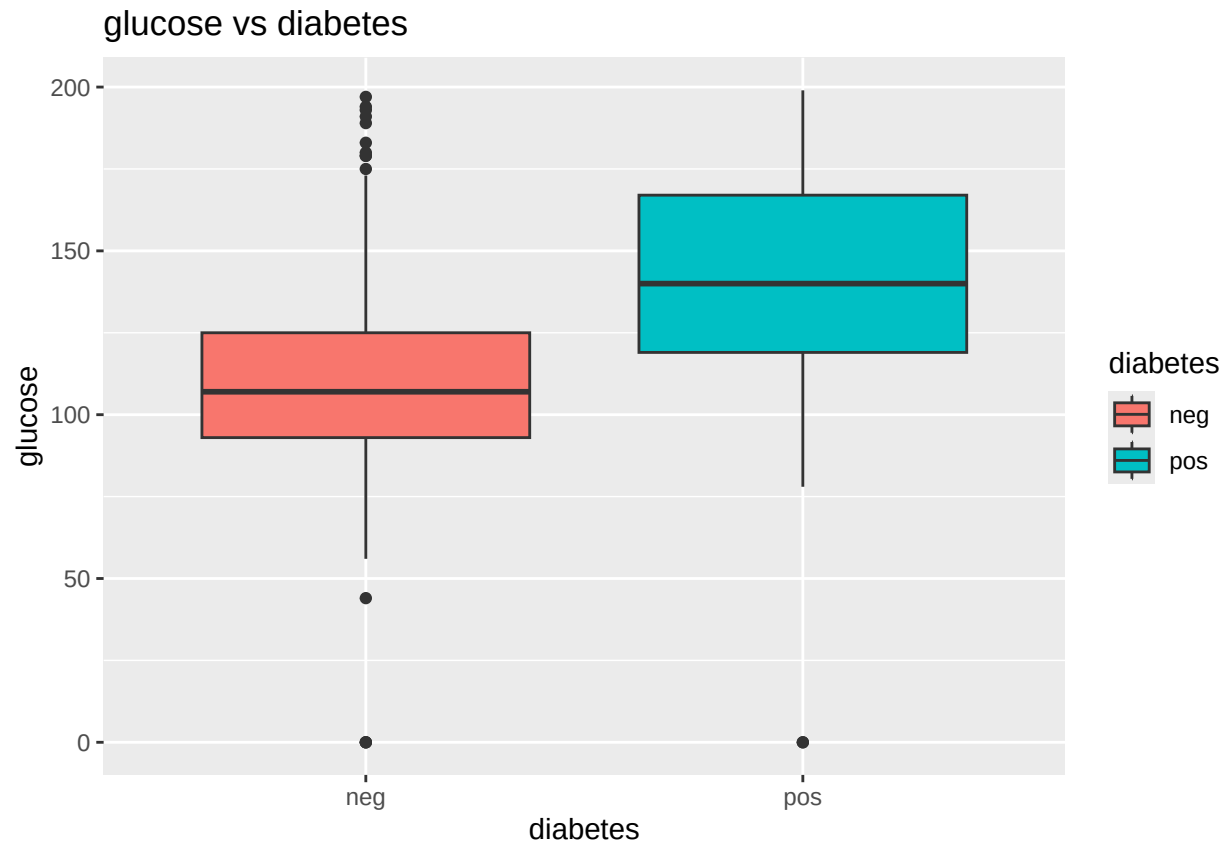


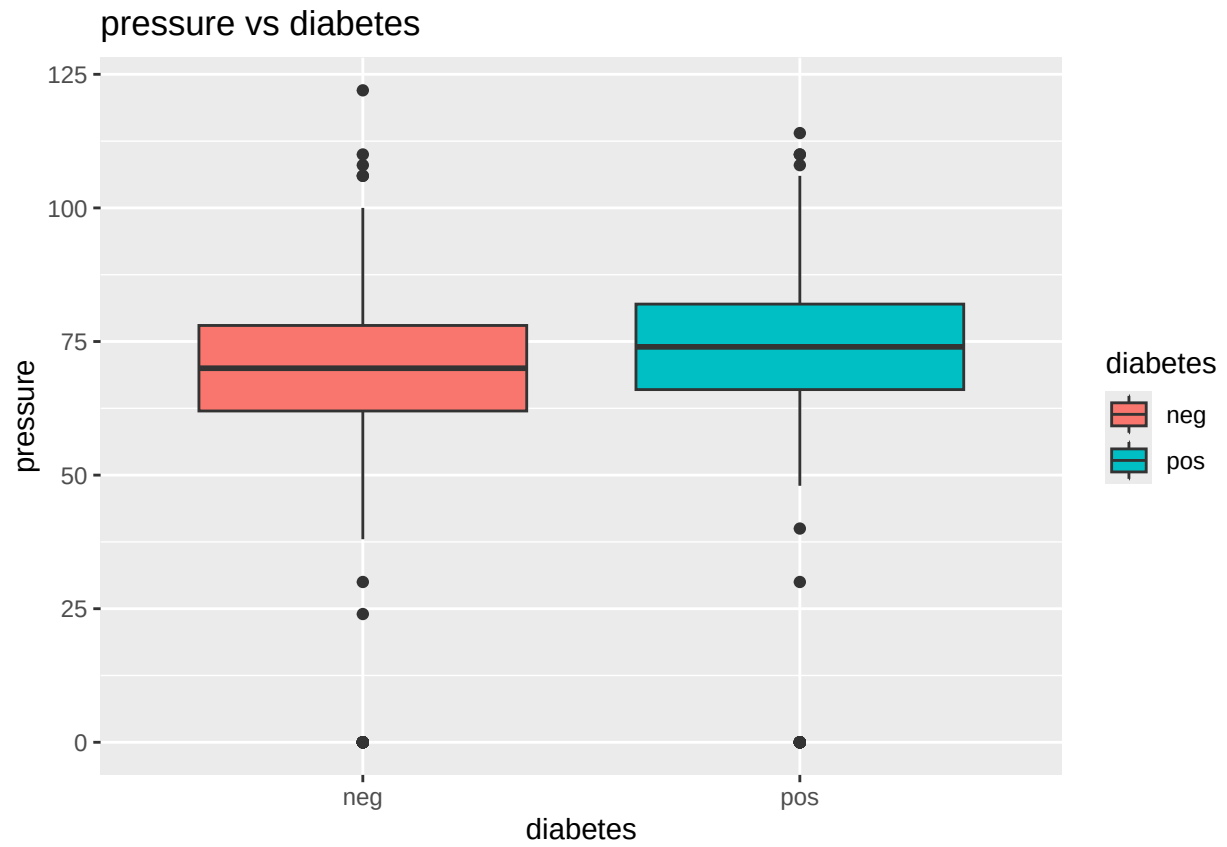
Histogramme de age

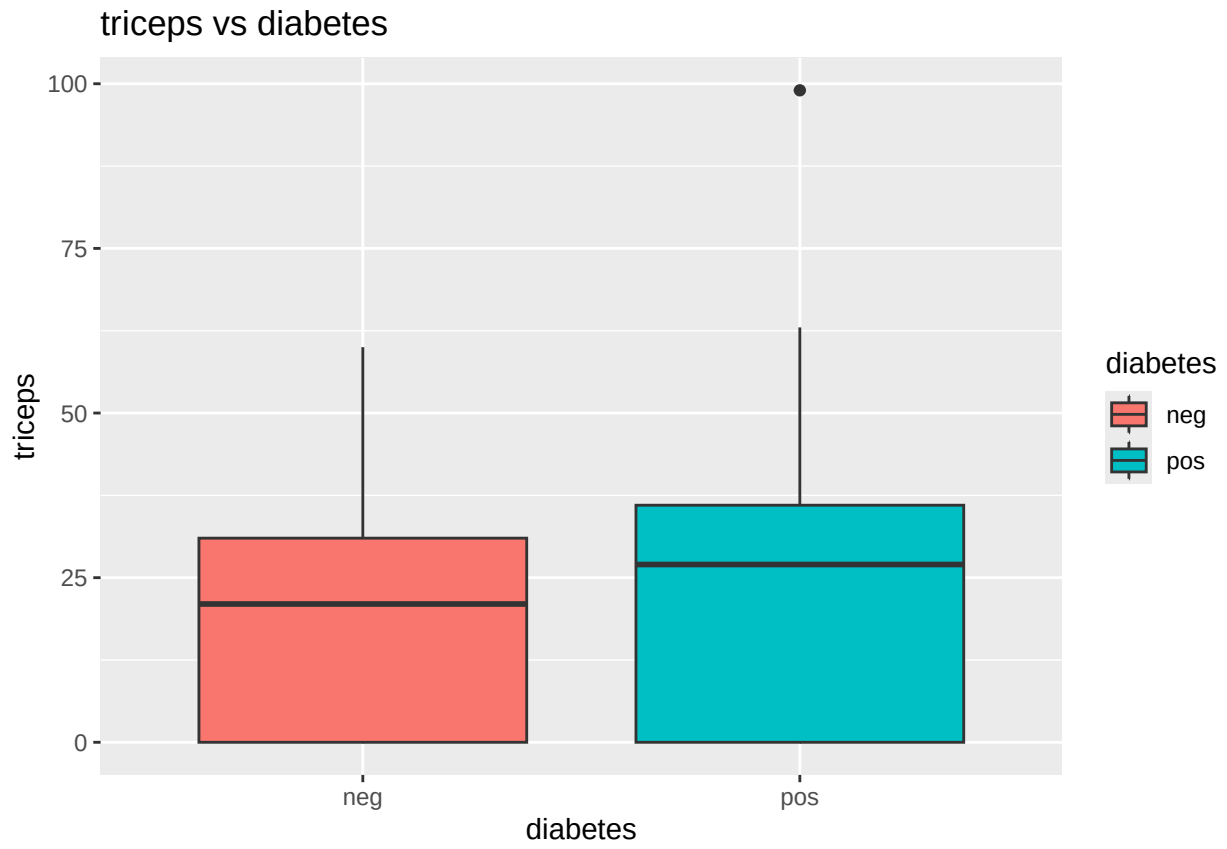


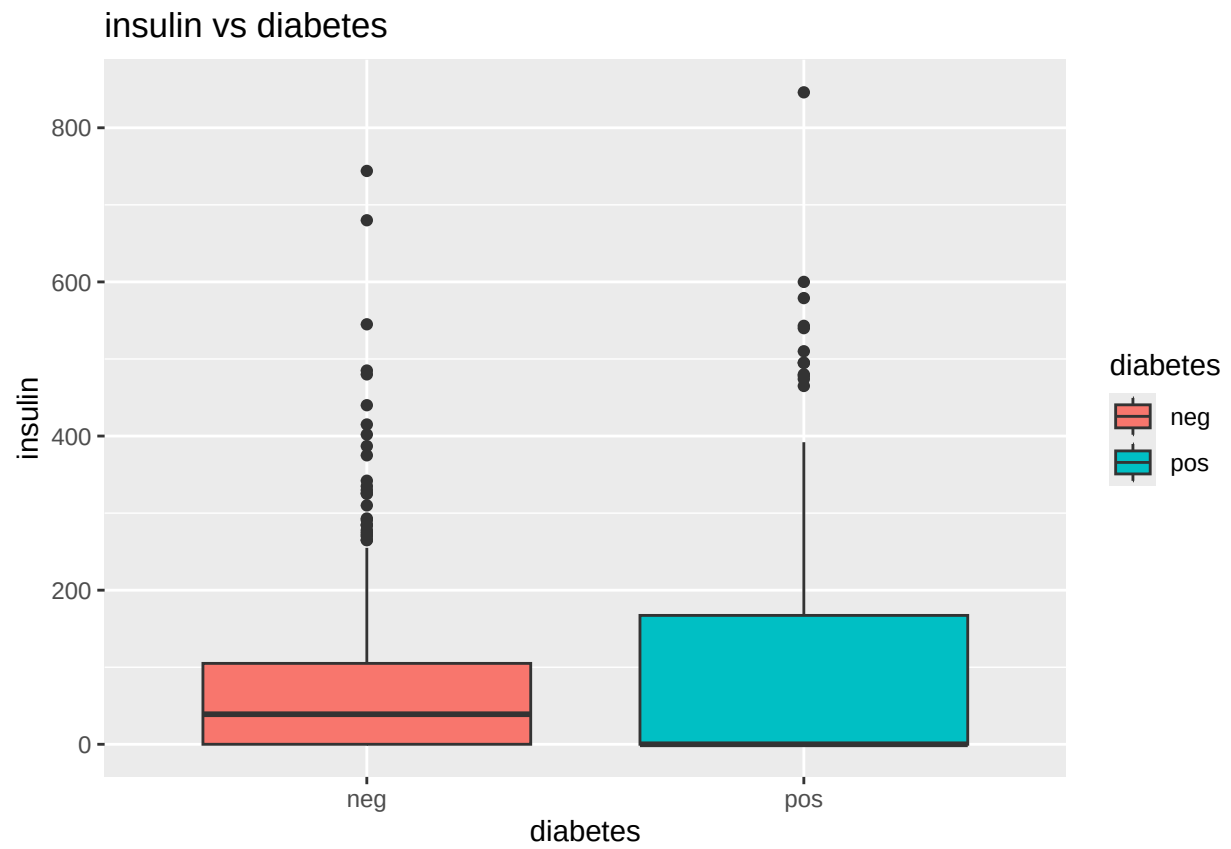
```
for (col in names(df)[1:8]) {  
  print(ggplot(df, aes(x = diabetes, y = .data[[col]], fill = diabetes)) +  
    geom_boxplot() +  
    ggtitle(paste(col, "vs diabetes")))  
}
```

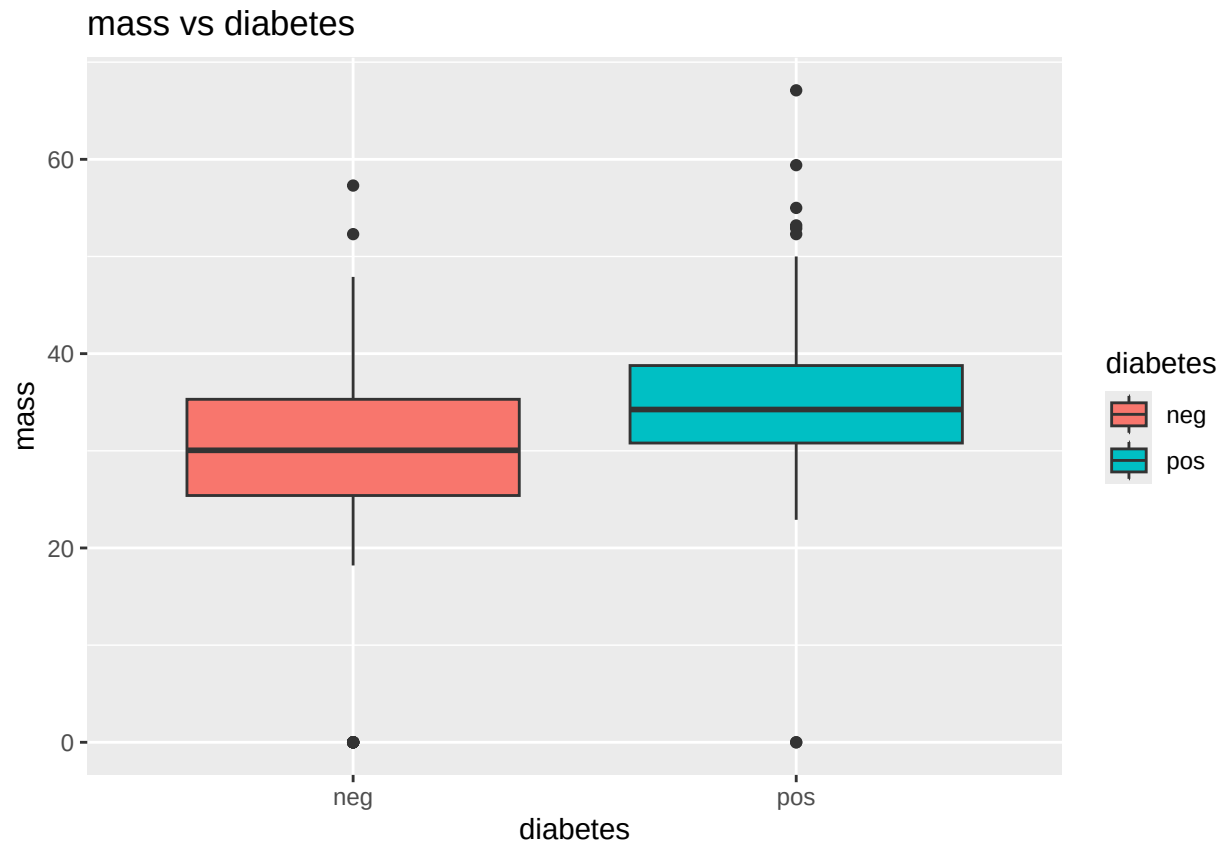


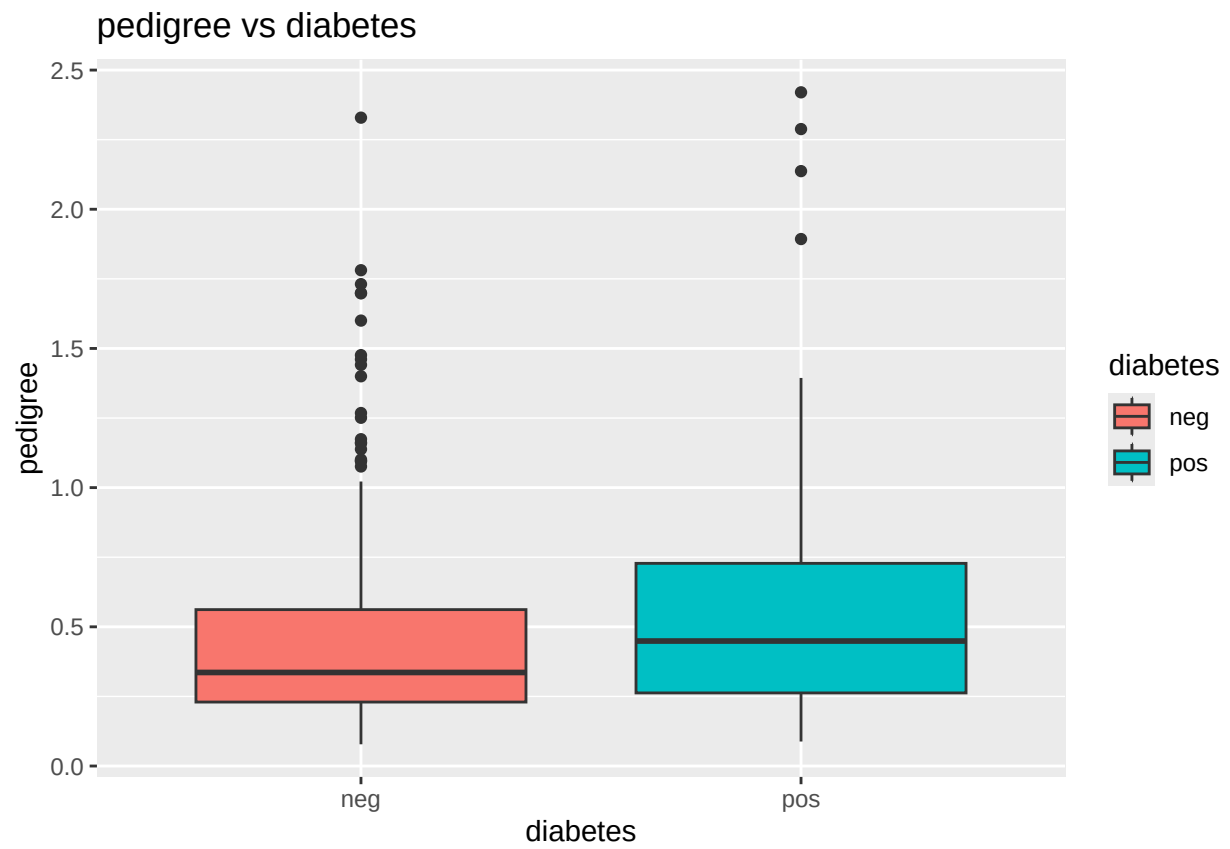


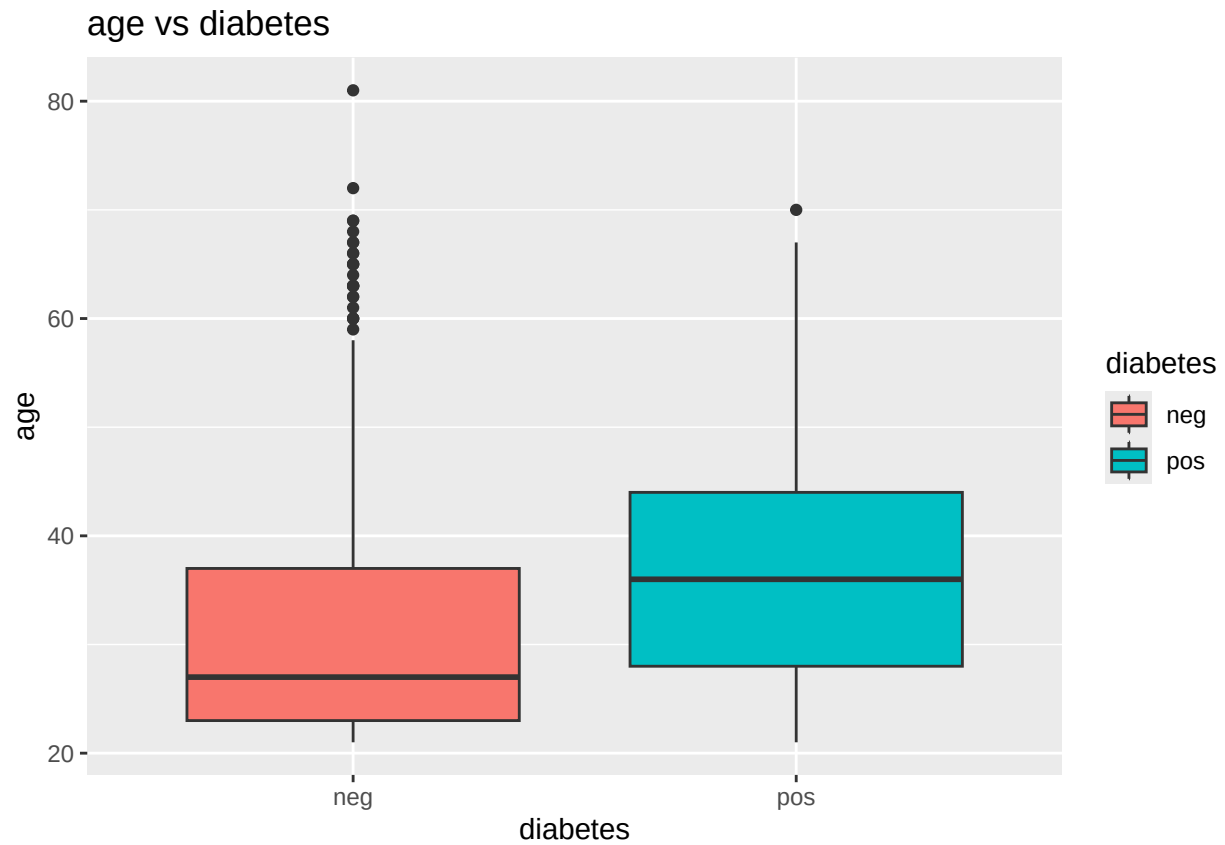












```
df_sample <- df[sample(nrow(df), 200), ]
ggpairs(df_sample[, c("glucose", "age", "mass", "diabetes")],
  aes(color=diabetes, alpha=0.6),
  upper = list(continuous = wrap("cor", size=3)))
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



Exploration non supervisée :

#transformation de nos variables

```
dfV2 =df
```

#on souhaite gerer les valeurs abérantes

#cas des 0 abérants

#ces 5 variables ne devraient pas pouvoir valoir 0

```
zeroA <- c("glucose","pression","triceps","insulin","mass")
```

#remplacement des 0

```
for (i in zeroA){
  dfV2[dfV2[[i]] == 0, i] <- NA
}
```

#mtn on peut calculer la medianne et la moyenne

#on va remplacer chaque 0 par la medianne pour completer

```
for (i in zeroA){
  medianne <- median(dfV2[[i]],na.rm =TRUE) #on retire les NA
  dfV2[is.na(dfV2[[i]]),i] <- medianne
}
```

```
library(dplyr)
```

```
X <- dfV2[, names(dfV2) != "diabetes"]
```

```
Y <- dfV2$diabetes
```

```

#ACP
#standardisation
Xstand <- scale(X)

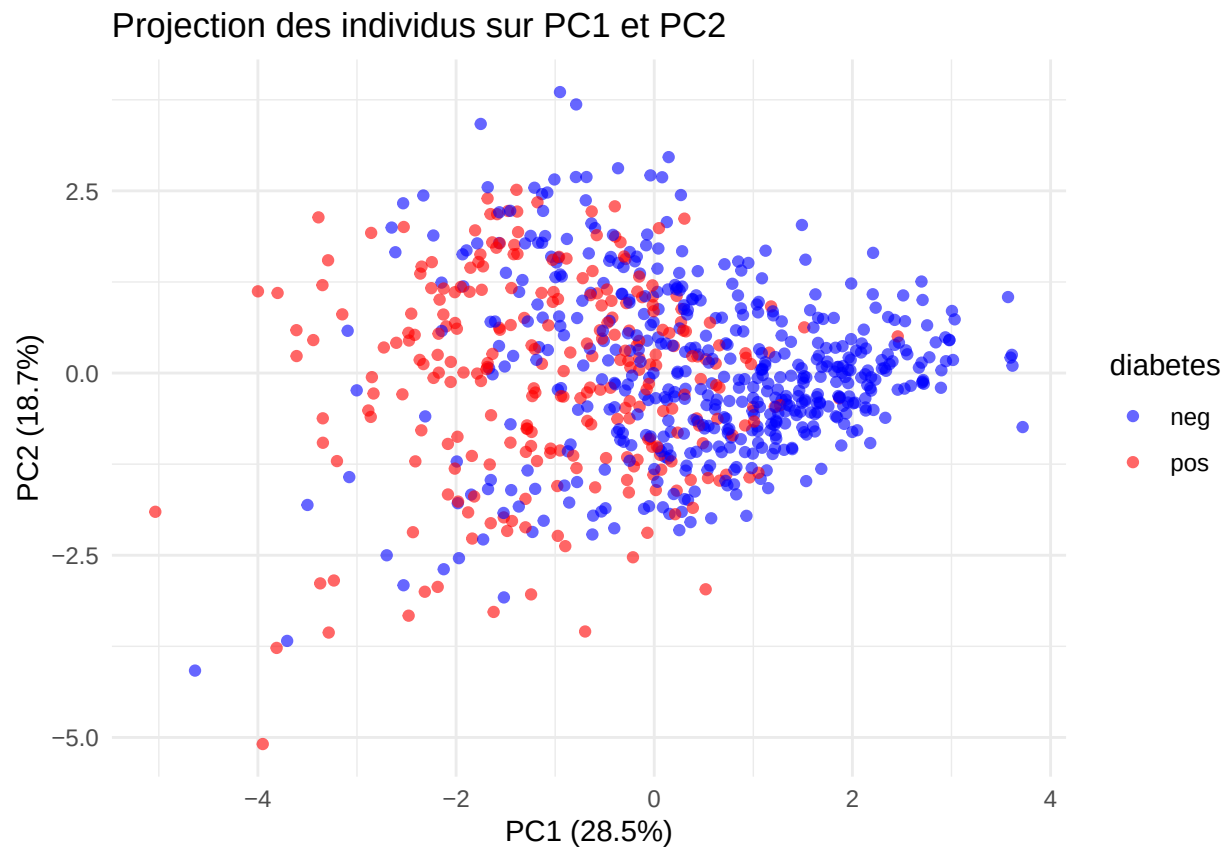
acp = prcomp(Xstand,center = FALSE, scale. = FALSE)

inertie_pourcentage <- acp$sdev^2 / sum(acp$sdev^2)
inertie_pourcentage_cumuler <- cumsum(inertie_pourcentage)

# Projection des individus
individus <- data.frame(
  PC1 = acp$x[,1],
  PC2 = acp$x[,2],
  diabetes = Y
)

ggplot(individus, aes(x = PC1, y = PC2, color = diabetes)) +
  geom_point(alpha = 0.6) +
  labs(title = "Projection des individus sur PC1 et PC2",
       x = paste0("PC1 (", round(inertie_pourcentage[1]*100, 1), "%)"),
       y = paste0("PC2 (", round(inertie_pourcentage[2]*100, 1), "%)")) +
  theme_minimal() +
  scale_color_manual(values = c("neg" = "blue", "pos" = "red"))

```

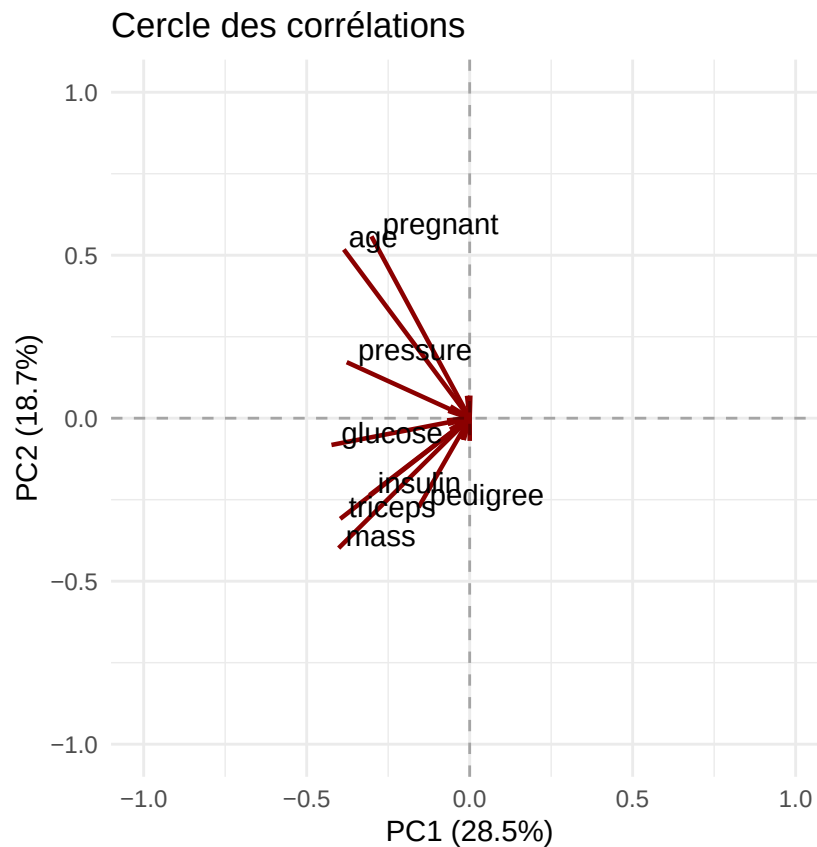


```

# Cercle des corrélations
variables <- data.frame(
  PC1 = acp$rotation[,1],
  PC2 = acp$rotation[,2],
  nom = rownames(acp$rotation)
)

ggplot(variables, aes(x = PC1, y = PC2)) +
  geom_segment(aes(xend = 0, yend = 0),
    arrow = arrow(length = unit(0.3, "cm")),
    color = "darkred", linewidth = 0.8) +
  geom_text(aes(label = nom), hjust = -0.1, vjust = -0.1, size = 4) +
  xlim(-1, 1) + ylim(-1, 1) +
  geom_hline(yintercept = 0, linetype = "dashed", alpha = 0.3) +
  geom_vline(xintercept = 0, linetype = "dashed", alpha = 0.3) +
  labs(title = "Cercle des corrélations",
    x = paste0("PC1 (", round(inertia_pourcentage[1]*100, 1), "%)"),
    y = paste0("PC2 (", round(inertia_pourcentage[2]*100, 1), "%)")) +
  theme_minimal() +
  coord_fixed()

```



```

#Kmeans

#methode du coude pour savoir combien de cluster on aura besoin
#scic pour somme des carrés intra classe

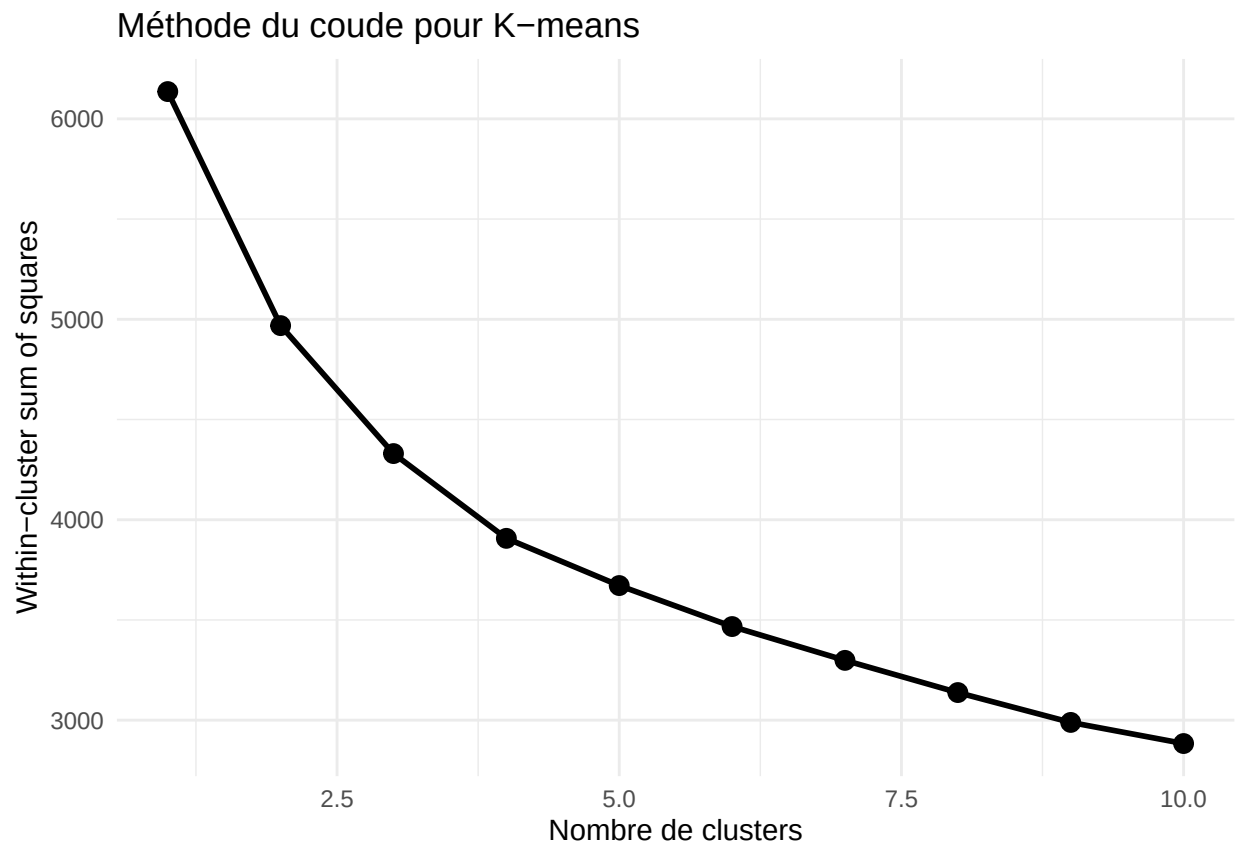
```



```
scic <- sapply(1:10, function(k) {
  kmeans(Xstand, centers = k, nstart = 20)$tot.withinss
})
```

#la méthode ne nous donne pas clairement un K optimal. J'ai donc décidé de tester le moment où il y a l
#on remarquera que K=2 est le meilleur choix même s'il sépare assez mal les 2 classes

```
data.frame(k = 1:10, WSS = scic) %>%
  ggplot(aes(x = k, y = WSS)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  theme_minimal() +
  labs(title = "Méthode du coude pour K-means",
       x = "Nombre de clusters",
       y = "Within-cluster sum of squares")
```



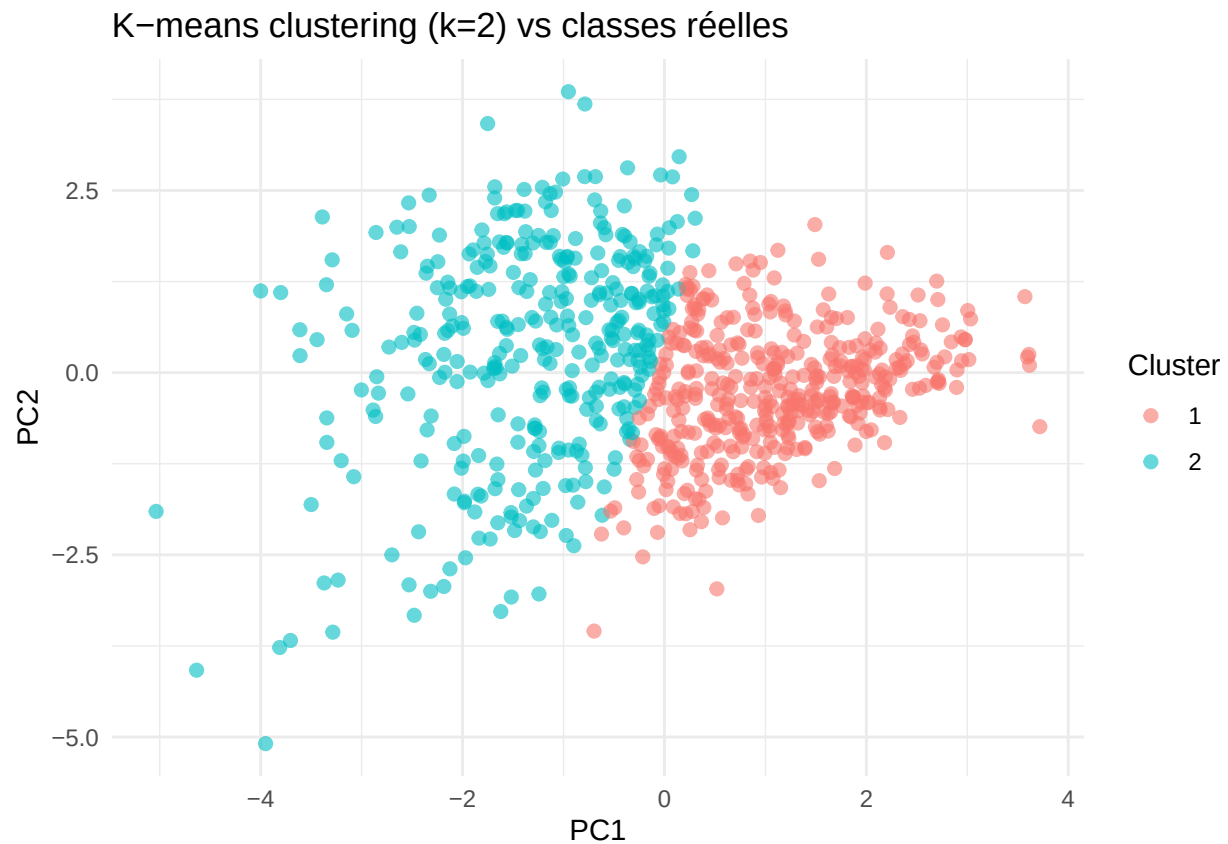
```
# K-means avec k=2
kmeans_result <- kmeans(Xstand, centers = 2, nstart = 25)
```

Visualisation sur les composantes principales

```
kmeans_df <- data.frame(
  PC1 = acp$x[,1],
  PC2 = acp$x[,2],
  Cluster = factor(kmeans_result$cluster),
  Diabetes = Y
```

```
)

ggplot(kmeans_df, aes(x = PC1, y = PC2, color = Cluster)) +
  geom_point(alpha = 0.6, size = 2) +
  theme_minimal() +
  labs(title = "K-means clustering (k=2) vs classes réelles")
```



```
# Confusion entre clusters et classes réelles
table(Cluster = kmeans_result$cluster, Diabetes = Y)
```

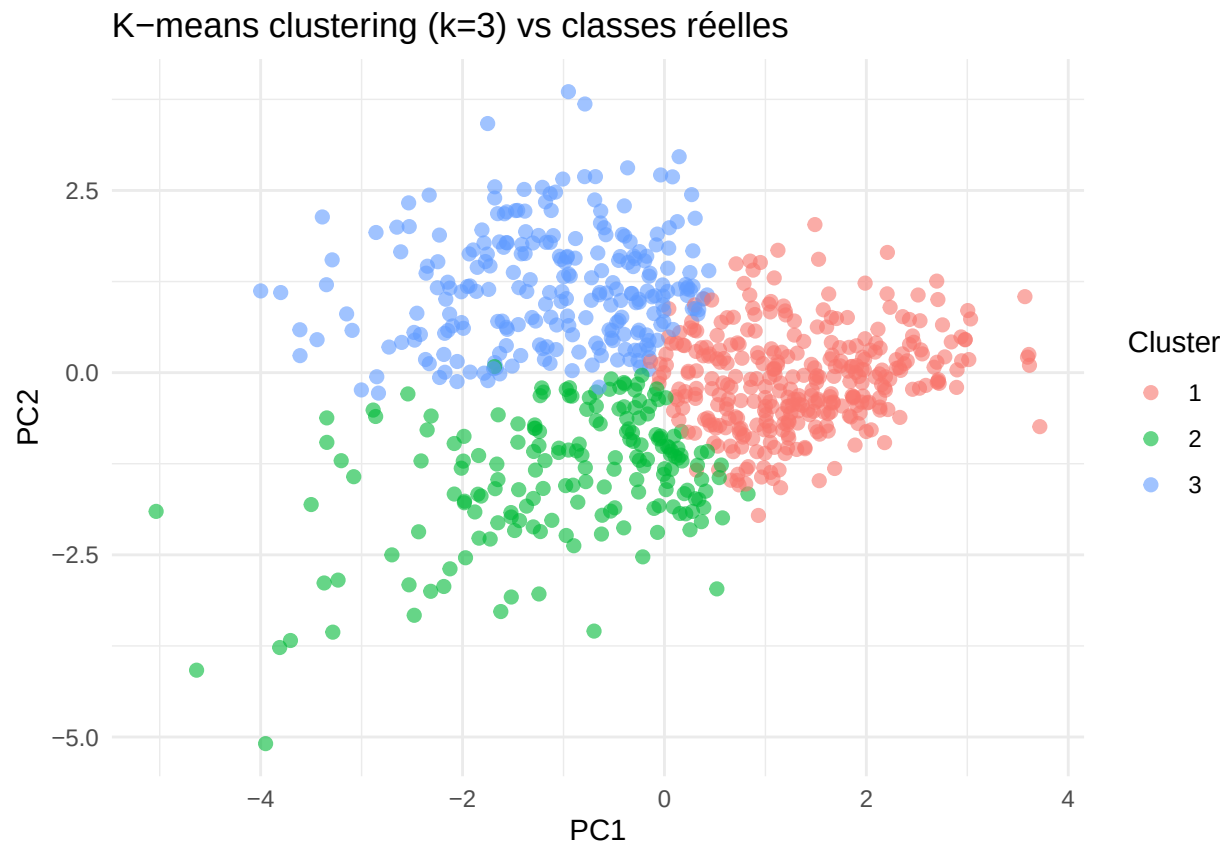
```
##      Diabetes
## Cluster neg pos
##      1 343  73
##      2 157 195
```

```
# K-means avec k=3
kmeans_result <- kmeans(Xstand, centers = 3, nstart = 25)

# Visualisation sur les composantes principales
kmeans_df <- data.frame(
  PC1 = acp$x[,1],
  PC2 = acp$x[,2],
  Cluster = factor(kmeans_result$cluster),
  Diabetes = Y
```

```
)

ggplot(kmeans_df, aes(x = PC1, y = PC2, color = Cluster)) +
  geom_point(alpha = 0.6, size = 2) +
  theme_minimal() +
  labs(title = "K-means clustering (k=3) vs classes réelles")
```



```
# Confusion entre clusters et classes réelles
table(Cluster = kmeans_result$cluster, Diabetes = Y)
```

```
##      Diabetes
## Cluster neg pos
##      1 293  44
##      2  87  96
##      3 120 128
```

```
# K-means avec k=5
kmeans_result <- kmeans(Xstand, centers = 5, nstart = 25)

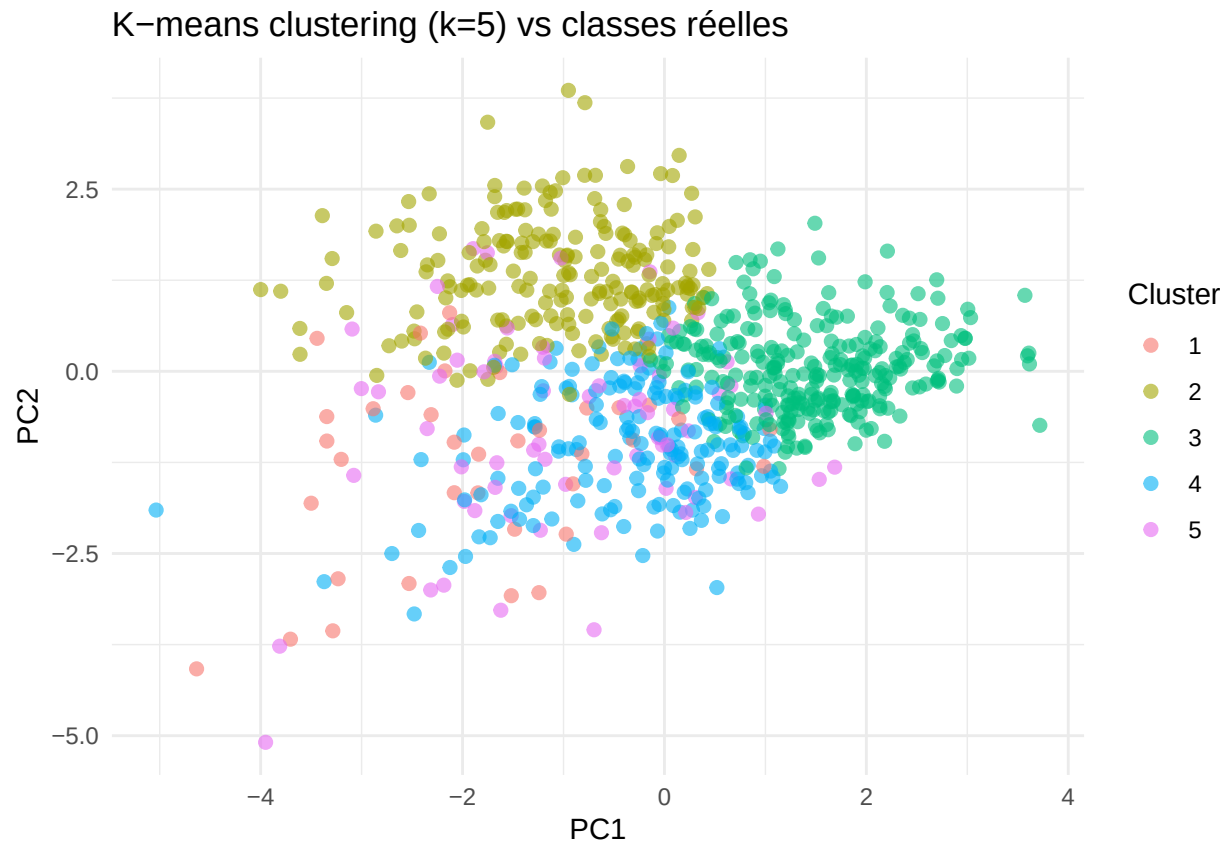
# Visualisation sur les composantes principales
kmeans_df <- data.frame(
  PC1 = acp$x[,1],
  PC2 = acp$x[,2],
  Cluster = factor(kmeans_result$cluster),
```

```

Diabetes = Y
)

ggplot(kmeans_df, aes(x = PC1, y = PC2, color = Cluster)) +
  geom_point(alpha = 0.6, size = 2) +
  theme_minimal() +
  labs(title = "K-means clustering (k=5) vs classes réelles")

```



```

# Confusion entre clusters et classes réelles
table(Cluster = kmeans_result$cluster, Diabetes = Y)

```

```

##      Diabetes
## Cluster neg pos
##      1  16  21
##      2 102 105
##      3 251  28
##      4 108  69
##      5  23  45

```

#on remarque que K=2 est le meilleur choix au vue des resultats

conclusion :

-le K-means K = 2 qui est pourtant le meilleur choix n'arrive pas a bien separer les 2 classes.

- Les classes ne sont pas parfaitement séparables dans l'espace des composantes principales, ce qui explique la difficulté de séparation.

-L'ACP démontre que les deux premières composantes expliquent une petite partie de la variance totale.

Prédiction avec un classifieur de Bayes naïf et une méthode de votre choix

```
# Division des données
part <- createDataPartition(dfV2$diabetes, p = 0.75, list = FALSE)
de <- dfV2[part, ]
dt <- dfV2[~part, ]

# Classifieur de Bayes Naïf
bayesNaive <- naiveBayes(diabetes ~ ., data = de)

bayesNaive_entr <- predict(bayesNaive, de)
bayesNaive_test <- predict(bayesNaive, dt)

cat("Entraînement")
```

```
## Entraînement
```

```
confusionMatrix(bayesNaive_entr, de$diabetes)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction neg pos
##      neg 313  76
##      pos  62 125
##
##              Accuracy : 0.7604
##              95% CI : (0.7234, 0.7947)
##      No Information Rate : 0.651
##      P-Value [Acc > NIR] : 9.199e-09
##
##              Kappa : 0.4641
##
##      McNemar's Test P-Value : 0.2685
##
##              Sensitivity : 0.8347
##              Specificity : 0.6219
##      Pos Pred Value : 0.8046
##      Neg Pred Value : 0.6684
##      Prevalence : 0.6510
##      Detection Rate : 0.5434
##      Detection Prevalence : 0.6753
##      Balanced Accuracy : 0.7283
##
##      'Positive' Class : neg
##
```

```
cat("Test")
```

```
## Test
```

```
nb_cm <- confusionMatrix(bayesNaive_test, dt$diabetes)
print(nb_cm)
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction neg pos
```

```
##           neg 102  29
```

```
##           pos   23  38
```

```
##
```

```
##           Accuracy : 0.7292
```

```
##           95% CI : (0.6605, 0.7906)
```

```
## No Information Rate : 0.651
```

```
## P-Value [Acc > NIR] : 0.01287
```

```
##
```

```
##           Kappa : 0.3913
```

```
##
```

```
## McNemar's Test P-Value : 0.48807
```

```
##
```

```
##           Sensitivity : 0.8160
```

```
##           Specificity : 0.5672
```

```
##           Pos Pred Value : 0.7786
```

```
##           Neg Pred Value : 0.6230
```

```
##           Prevalence : 0.6510
```

```
##           Detection Rate : 0.5312
```

```
## Detection Prevalence : 0.6823
```

```
##           Balanced Accuracy : 0.6916
```

```
##
```

```
##           'Positive' Class : neg
```

```
##
```

```
# Erreur de prédiction
```

```
erreur_entrainement <- mean(bayesNaive_entr != de$diabetes)
```

```
erreur_test <- mean(bayesNaive_test != dt$diabetes)
```

```
cat("Erreur d'entraînement", round(erreur_entrainement, 4), "\n")
```

```
## Erreur d'entraînement 0.2396
```

```
cat("Erreur de test", round(erreur_test, 4), "\n")
```

```
## Erreur de test 0.2708
```

```
#Je choisis la méthode de la foret aléatoire
```

```
#install.packages("randomForest")
```

```
library(randomForest)
```

```

## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.

##
## Attachement du package : 'randomForest'

## L'objet suivant est masqué depuis 'package:dplyr':
##
##      combine

## L'objet suivant est masqué depuis 'package:ggplot2':
##
##      margin

# Random Forest
set.seed(123)
foretAleatoire <- randomForest(diabetes ~ ., data = de,
                               ntree = 500, mtry = 3, importance = TRUE)

foretAleatoire_entr <- predict(foretAleatoire, de)
foretAleatoire_test <- predict(foretAleatoire, dt)

cat("Matrice de confusion - Test (Random Forest):\n")

## Matrice de confusion - Test (Random Forest):

rf_cm <- confusionMatrix(foretAleatoire_test, dt$diabetes)
print(rf_cm)

## Confusion Matrix and Statistics
##
##              Reference
## Prediction neg pos
##      neg 109  28
##      pos  16  39
##
##              Accuracy : 0.7708
##              95% CI : (0.7048, 0.8283)
##      No Information Rate : 0.651
##      P-Value [Acc > NIR] : 0.0002216
##
##              Kappa : 0.4738
##
##      McNemar's Test P-Value : 0.0972544
##
##              Sensitivity : 0.8720
##              Specificity : 0.5821
##      Pos Pred Value : 0.7956
##      Neg Pred Value : 0.7091
##              Prevalence : 0.6510
##      Detection Rate : 0.5677

```

```
## Detection Prevalence : 0.7135
## Balanced Accuracy : 0.7270
##
## 'Positive' Class : neg
##
```

```
# Erreur de prédiction
```

```
erreur_entrainement_rf <- mean(foretAleatoire_entr != de$diabetes)
erreur_test_rf <- mean(foretAleatoire_test != dt$diabetes)

cat("Erreur d'entraînement", round(erreur_entrainement_rf, 4), "\n")
```

```
## Erreur d'entraînement 0
```

```
cat("Erreur de test:", round(erreur_test_rf, 4), "\n")
```

```
## Erreur de test: 0.2292
```

```
# Importance des variables
```

```
importance_df <- data.frame(
  Variable = rownames(foretAleatoire$importance),
  Importance = foretAleatoire$importance[, "MeanDecreaseGini"]
) %>% arrange(desc(Importance))

ggplot(importance_df, aes(x = reorder(Variable, Importance), y = Importance)) +
  geom_bar(stat = "identity", fill = "darkgreen") +
  coord_flip() +
  theme_minimal() +
  labs(title = "Importance des variables (Random Forest)",
       x = "Variable",
       y = "impact des variables si la minimisation de l'impureté")
```